

User Manual

Platform Version: 0.0.4

Author: Meow Li

Copyright: Copyright (c) 2026 Meow Li. All Rights Reserved.

This manual explains how users should think about Partner and how advanced users will eventually author systems and gadgets. It is not a backlog. Planned or missing work belongs in ``docs/todo.md``.

1. Product Layers

Partner has two product layers.

The first layer is **system authoring and execution**. Users define bidding systems, edit gadgets, share system files, generate system notes, and ask the engine what a bot partner should bid.

The second layer is **tournament and inter-user play**. Users bring their systems into human-plus-custom-bot competition, challenges, sharing, and analytics.

The current project is focused on the first layer. Tournament features are designed as a later layer on top of the system engine.

2. Basic Notation

Partner uses compact canonical bidding notation.

P = Pass
X = Double
R = Redouble
C = Clubs
D = Diamonds
H = Hearts
S = Spades
N = Notrump

Contract calls combine a level and a suit:

1C 1D 1H 1S 1N ... 7N

The platform stores notrump as ``N``, so ``1NT`` is normalized to ``1N``.

Seats are absolute and lowercase:

n e s w

Vulnerability values are:

none ns ew both

3. Asking The Engine For A Bid

A bid request needs five kinds of information:

- ? selected system,
- ? seat,
- ? auction,
- ? hand,
- ? environment.

Current backend example:

```
{
  "system": {"id": "expert_2over1"},
  "seat": "n",
  "auction": "1NP2DP",
  "hand": "SAQ74HKJ83DA62CQ5",
  "environment": {
    "dealer": "n",
    "vulnerability": "none",
    "scoring": "IMP"
  }
}
```

The current sample returns `3H` because the auction shows a heart transfer and the hand qualifies for the current superaccept rule.

4. Hand Strings

Raw hand input uses one compact string. Dictionary-shaped hand input is not a user input format.

Example:

H8763SK10C2DAKQ987

This means:

H 8763
S K10
C 2
D AKQ987

Rules:

- ? Suit markers are `S H D C`.
- ? Suits can appear in any order.
- ? A suit can appear more than once; sections are concatenated.
- ? `10` and `T` both mean ten.
- ? `x` means an unknown small card placeholder.
- ? `` marks a void.
- ? A complete raw hand should contain 13 cards.

Invalid hands should fail clearly, including wrong card count, repeated known cards, unknown symbols, rank text before any suit marker, and a void marker mixed with cards.

5. Reading An Explanation

Partner separates explanation into two layers.

****Public meaning**** describes what the call means as a partnership agreement. This is the disclosure-oriented layer.

****Internal origin**** describes how the engine chose the call. This is for training, debugging, and system design. It may include candidate comparisons that do not belong in opponent disclosure.

When reviewing an engine result, inspect:

- ? selected call,
- ? public meaning,
- ? gadget origin,
- ? rule origin,
- ? compared candidates,
- ? structured criteria,
- ? diagnostics.

Diagnostics indicate that the system may be incomplete, ambiguous, or missing an interpretation for part of the auction.

6. Systems And Gadgets

A ****system**** is a collection of active gadgets and settings.

A ****gadget**** is a portable bidding module. A gadget may define meanings, selection rules, semantic facts, alertability, and later continuations or constraints.

Users may see the term ****basic system**** or ****base system**** for foundational agreements such as 2/1 Game Forcing. Technically, a base system is still represented as an ordinary gadget. It is a user-facing label for rules that cover basic openings and early auction structure.

Gadget files live under:

```
backend/gadgets/
```

System files live under:

```
backend/systems/
```

Human-authored files should use real YAML. JSON-compatible YAML is accepted for generated files and tooling, but real YAML is the recommended style.

Future authoring will add a formal Bridge System Language and GUI forms. YAML is currently the practical executable rule format. Long term, users may choose between guided forms, BSL text, and advanced IR/YAML editing, but every path must validate to the same executable rule representation before the engine uses it.

7. Gadget Directory Shape

A gadget is stored as one directory. The metadata lives in `gadget.yaml`, and the rules can be split across multiple YAML files.

```
# backend/gadgets/four_way_jacoby_transfer/gadget.yaml
id: four_way_jacoby_transfer
namespace: notrump_response
name: Four-Way Jacoby Transfer
version: 0.1.0
author:
  name: Partner Prototype
```

Rule files use a `rules` list:

```
# backend/gadgets/four_way_jacoby_transfer/notrump_responses.yaml
rules:
- id: transfer_hearts_2D
  context:
    auction_pattern: "1NP"
    seat_positions: [1, 2, 3, 4]
  call: 2D
  selection:
    applicability:
      self.hearts:
        min: 5
    algorithm: weighted_score
    criteria:
      - criterion_id: jacoby_transfer_heart_length
        evaluator: min_value
        input: self.hearts
        min: 5
        weight: 60
  meaning:
    call_nature: artificial
    action_type: transfer
    target_suit: H
    alertable: true
  semantic_effects:
    - fact_type: transfer
      target_suit: H
      status: pending
```

Continuation rules can live in another YAML file:

```
# backend/gadgets/four_way_jacoby_transfer/continuations_hearts.yaml
rules:
- id: superaccept_hearts
  context:
    auction_pattern: "1NP2DP"
    seat_positions: [1, 2, 3, 4]
  call: 3H
  selection:
    algorithm: weighted_score
    criteria:
      - criterion_id: formal_heart_transfer_state
        evaluator: fact_exists
        query:
          fact_type: transfer
          target_suit: H
          status: pending
        weight: 40
      - criterion_id: four_card_heart_support
        evaluator: min_value
```

```

        input: self.hearts
        min: 4
        weight: 30
    meaning:
        call_nature: conventional
        action_type: superaccept
        target_suit: H
        alertable: true

```

This example shows the essential rule content:

? `context.auction\_pattern` is the machine auction pattern.

? `context.seat\_positions` expands the pattern by adding the initial passes for those seats. For example, `auction\_pattern: "1HP"` with `seat\_positions: [3, 4]` covers `PP1HP` and `PPP1HP`.

? `selection` says when the engine should choose the call with the current hand.

? `meaning` says what the call means as a partnership agreement.

? `semantic\_effects` create machine-readable facts used by later rules.

For a Jacoby transfer, the selection rule must say when the bot should choose the transfer. For example, a heart transfer through `2D` should include a structured condition such as `self.hearts >= 5`. The meaning rule then says what `2D` means when it appears in the auction.

A rule is a unified object. It can contain selection logic, public meaning, and semantic effects together.

8. System Files

A system file chooses active gadgets:

```

id: expert_2over1
name: Expert 2/1
gadgets:
  - two_over_one
  - four_way_jacoby_transfer

```

The order is useful for stable loading, but the engine searches active gadget rules globally. After `1NP`, the `1N` opening can come from the `two\_over\_one` gadget while the transfer response can come from `four\_way\_jacoby\_transfer`.

9. Authoring Principles

Gadget authors should follow these principles:

? Use short, readable IDs.

? Keep one gadget per directory.

? Put the gadget ID in the directory name.

? Put public partnership disclosure in `meaning`.

? Put machine-readable auction consequences in `semantic\_effects`.

? Use structured criteria instead of prose as the source of selection reasoning.

? Mark alertability explicitly until automated alert analysis exists.

? Treat LLM-generated gadgets as drafts until reviewed and tested.

? Use top-level `applicability` for context conditions that also matter when replaying past calls.

? Use `selection.applicability` and criteria for current-hand bid selection.

? Use the shared semantic ontology for machine-readable effects. Advanced concepts such as transfers, agreed suits, forcing status, competitive interference, and keycard asks should be represented as formal state, not private prose labels.

The detailed ontology is maintained in:

```
docs/semantic_ontology.md
```

10. System Notes Workflows

Partner should eventually support two directions.

From structured files to human notes:

```
system/gadget files -> readable system notes
```

From human notes to draft files:

```
human system notes -> LLM-assisted draft -> BSL or IR -> validation -> user approval -> gadget files
```

The structured files remain the source of truth.

Expected authoring surfaces:

? Form view for common agreements and non-programmer editing.

? BSL view for formal bridge-like source text.

? IR/YAML view for advanced inspection, debugging, and exact executable structure.

The engine executes validated IR. BSL and GUI forms are authoring layers over that executable model.

11. Troubleshooting Concepts

If the engine returns no bid, common causes are:

? no rule matches the auction,

? a needed semantic fact was never created,

? the system did not import the needed gadget,

? the auction is outside the implemented rule set.

If a historical call is undefined, add or fix a `meaning` rule for that auction position.

If two calls tie on score, the current engine reports ambiguity.

If no current selection rule matches, the current loaded default policy is to pass and emit a diagnostic. Later versions should make default behavior customizable, such as signing off, returning to an agreed suit, or bidding game when a game force has already been inferred.

12. Hand And Deal Workspace

The system layer should include a workspace for testing agreements on real or generated deals.

Expected capabilities:

- ? enter one hand manually,
- ? enter a full deal manually,
- ? deal random hands,
- ? import deal files such as PBN,
- ? choose system, seat, dealer, vulnerability, and scoring,
- ? step through the auction,
- ? ask the engine for the next bid,
- ? inspect public meaning, internal origin, candidates, and diagnostics.

This workspace is for system testing, training, and gadget debugging. It does not require tournament or account features.

13. Test Case Workflow

Project behavior examples live in editable YAML files under ``backend/tests/cases/``.

Add bidding examples to:

```
backend/tests/cases/bidding.yaml
```

Add pattern-matching examples to:

```
backend/tests/cases/matcher.yaml
```

Add compact-hand parser examples to:

```
backend/tests/cases/hands.yaml
```

The Python test runner loads these files automatically. Users and system authors can add practical examples as text cases without editing Python test code.

The readable companion document is:

```
backend/tests/test_cases.md
```

Update that document whenever the YAML test cases change.